MyOwnCompiler: Complete Compiler Engineering Roadmap

This roadmap is designed as a long-term compiler engineering project. The goal is not merely to build a parser, but to progressively construct a real language toolchain: Lexer → Parser → AST → Semantic Analyzer → Optimizer → Bytecode Generator → Virtual Machine → Advanced Features. Estimated pace: ~1 hour per day.

Phase 0 - Foundations (Week 1)

• Set up repository structure and build system.

• Define Token and TokenType abstractions.

• Learn lexical analysis concepts.

• Implement lexer for identifiers, numbers, operators, and keywords.

• Create lexer test suite and debugging utilities.

• Milestone: tokenize complete source files.

Phase 1 - Parsing and AST (Week 2)

• Study grammars, recursive-descent parsing, precedence handling.

• Parse arithmetic expressions.

• Implement parenthesis support.

• Build AST node hierarchy.

• Create AST pretty printer and visualization tools.

• Milestone: source code → AST.

Phase 2 - Language Constructs (Week 3)

• Variable declarations and assignments.

• Blocks and nested scopes.

• If statements.

• While loops.

• Statement parsing framework.

• Milestone: simple programs represented in AST.

Phase 3 - Semantic Analysis (Week 4)

• Symbol table implementation.

• Scope tracking.

• Undefined variable detection.

• Duplicate declaration detection.

• Type checking preparation.

• Milestone: semantic validation.

Phase 4 - Bytecode Generation (Week 5)

• Design bytecode instruction set.

• Create intermediate representation.

• Generate instructions from AST.

• Arithmetic and variable operations.

• Milestone: AST → executable bytecode.

Phase 5 - Virtual Machine (Week 6)

• Design stack machine.

• Implement instruction dispatcher.

• Memory model for variables.

• Program execution.

• Milestone: first executable language.

Phase 6 - Control Flow (Week 7)

• Jump instructions.

• Conditional branching.

• If execution.

• Loop execution.

• Nested control flow.

• Milestone: real program execution.

Phase 7 - Optimization (Week 8)

• Constant folding.

• Constant propagation.

• Dead code elimination.

• Strength reduction.

• Optimization pipeline architecture.

• Milestone: optimized bytecode.

Phase 8 - Advanced Compiler Engineering (Month 3+)

• Functions and call stack.

• Arrays and strings.

• User-defined types.

- Garbage collection.
- LLVM IR backend.
- Optional JIT compilation.
- Milestone: portfolio-grade compiler.

Detailed Daily Execution Strategy

Day 1: Repository structure, CMake setup, TokenType, Token design.

Day 2: Lexer skeleton and source traversal.

Day 3: Numbers, identifiers, keywords.

Day 4: Operators and punctuation.

Day 5: Lexer tests and file tokenization.

Day 6: Grammar study and parser architecture.

Day 7: Expression parsing.

Day 8: Precedence handling.

Day 9: Parentheses parsing.

Day 10: AST visualization.

Day 11: Variable declarations.

Day 12: Assignments.

Day 13: Blocks.

Day 14: If statements.

Day 15: While loops.

Day 16: Symbol table.

Day 17: Undefined variable errors.

Day 18: Duplicate declarations.

Day 19: Nested scope resolution.

Day 20: Semantic testing.

Day 21: Bytecode design.

Day 22: Instruction set.

Day 23: Code generation.

Day 24: Arithmetic bytecode.

Day 25: Variables in bytecode.

Day 26: VM stack.

Day 27: Instruction execution.

Day 28: Arithmetic runtime.

Day 29: Variable runtime.

Day 30: First executable program.

Stretch Goals

1. Static type system. 2. User-defined structs. 3. Module system. 4. REPL. 5. Register-based VM. 6. SSA Intermediate Representation. 7. LLVM backend. 8. WebAssembly backend. 9. Debugger integration. 10. Compiler benchmarking suite.